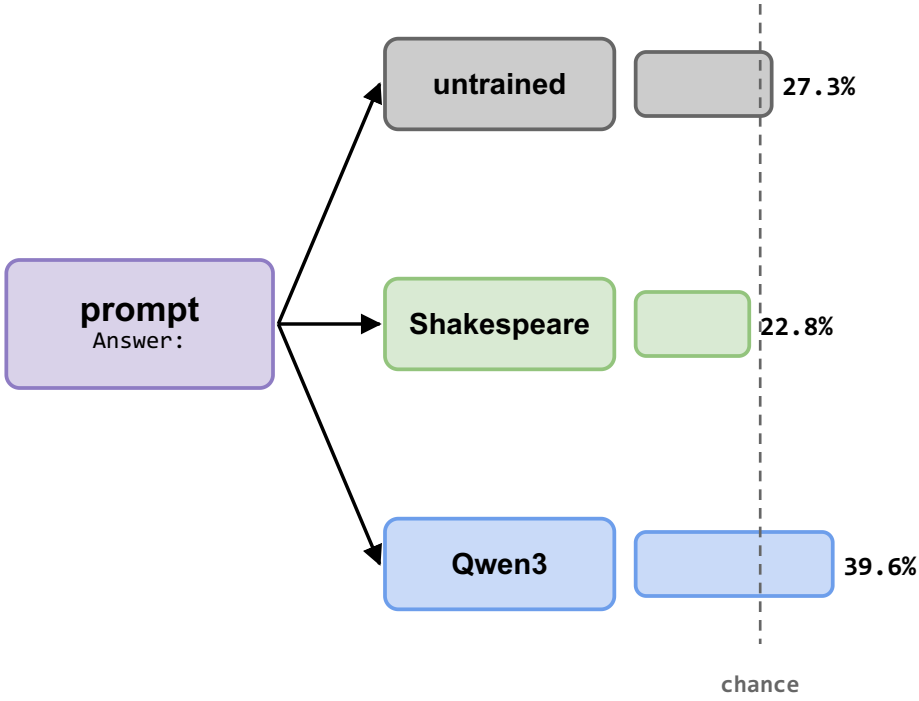


UNIT 02 · CHAPTER 1

Evaluating LLMs

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Validation Loss

Last chapter we finally trained irishQwen. We cut all of Shakespeare into about `2.5` million tokens, held the end of the corpus back, made five passes over the rest, and kept whichever epoch did best on that held-out text. The number we watched the whole time was the **validation loss**, the cross-entropy on text the model never trained on.

It went down, a lot, and the samples at the end of training looked a bit like Shakespeare. The obvious next question is whether irishQwen is any good. Before we can answer that we need a way to compare it with other models, starting with a version of the loss that doesn't depend on the tokenizer.

A loss is measured per token, and tokens are different sizes for different tokenizers (one of Qwen3's tokens is often a whole word, one of irishQwen's is often a couple of letters). Comparing losses per token across those two would be comparing apples and pieces of apple.

Bits per Character

So we convert the loss into **bits per character**: how many yes/no questions, on average, a model needs to pin down the next character of the text. Characters are the same for every tokenizer. A model that knew nothing and spread its bets evenly over all `512` of irishQwen's tokens would need `9` bits per token:

```python
import torch

# a uniform guess spreads its bets over all 512 tokens
bits = torch.log2(torch.tensor(512.0)).item()
print(bits, "bits per token")
```

OUTPUT

```
9.0 bits per token
```

An irishQwen token is about two characters of Shakespeare, so that uniform guess works out to `4.63` bits per character. Our untrained irishQwen, with its random weights, does worse than that at `5.40` (random weights aren't just ignorant, they're confidently wrong). After training on Shakespeare it needs only `1.72`.

That's a big improvement, but **it only tells us irishQwen got better at continuing Shakespeare**, not whether it knows anything. For that we need to ask it questions.

Benchmarks

If we want to know whether a student knows something, we give them an exam. The same idea works for models: a fixed set of questions with known answers, asked the same way of every model, scored the same way. We call that a **benchmark**, and running a model on one an **eval**.

The most famous one is **MMLU** (Massive Multitask Language Understanding, Hendrycks et al., 2020). It has about 14,000 test questions across 57 subjects, from high school macroeconomics to professional law, every one multiple choice with four options. Guessing gets 25%.

Here is one of the 112 machine learning questions:

Another term for out-of-distribution detection is?	
A. anomaly detection	B. one-class detection
C. train-test mismatch robustness	D. background detection

The answer is A. Everything in this chapter is measured on a sample of 399 MMLU questions, this one included.

Notice that nothing about a benchmark is specific to a chatbot. Our Shakespeare irishQwen can take the same exam as a model trained on half the internet. The first problem is that **a model doesn't read a question, it reads tokens**.

From Dataset Row to Prompt

On disk, our question is just line **48** of a file called `machine_learning_test.csv`, one comma-separated row holding the question, the four choices and the answer letter:

```
Another term for out-of-distribution detection is?,
anomaly detection,one-class detection,
train-test mismatch robustness,background detection,A
```

A model can't do much with that. Something has to turn the row into the kind of text the model reads, and decide what to leave out. That something is a **prompt template**. The one we'll use comes from EleutherAI's **lm-evaluation-harness**, the most common open tool for running benchmarks, and it writes MMLU questions like this:

```
The following are multiple choice questions (with
answers) about {subject}.

{question}
A. {A}
B. {B}
C. {C}
D. {D}
Answer:
```

The subject (machine learning) comes from the file's name. **The answer column is held back** until it's time to grade.

Prompt Templates

Filling in the template is just string building:

```
row = ("Another term for out-of-distribution detection"
       " is?,anomaly detection,one-class detection,"
       "train-test mismatch robustness,"
       "background detection,A")
*fields, answer = row.split(",")
question, choices = fields[0], fields[1:]

prompt = ("The following are multiple choice questions"
          " (with answers) about machine learning.\n\n")
prompt += question + "\n"
for letter, choice in zip("ABCD", choices):
    prompt += f"{letter}. {choice}\n"
prompt += "Answer:"

print(prompt)
print(len(prompt), "characters, held back:", answer)
```

OUTPUT

```
The following are multiple choice questions (with answers) about machine

Another term for out-of-distribution detection is?
A. anomaly detection
B. one-class detection
C. train-test mismatch robustness
D. background detection
Answer:
244 characters, held back: A
```

The text above, all `244` characters of it, is exactly what goes to the tokenizer. Every model we test will see these same characters (the tokens will differ). The prompt ends on `Answer:` with nothing after it, which leaves the model one obvious thing to do next.

Scoring the Next Token

We could let the model write a reply and then try to find a letter in it. Instead, the harness never lets it write anything at all. It runs the prompt through the model once and reads off the probability of the very next token being `" A"`, `" B"`, `" C"` or `" D"` (with the space, since after `Answer:` that's how the letter would actually appear). **Whichever letter is most likely is the model's answer.**

That only works cleanly if each letter is a single token. For both of our tokenizers it is:

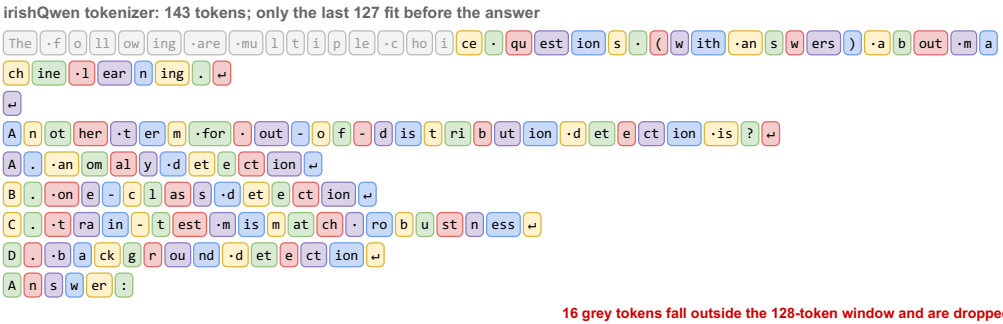
next token	Qwen3 id	irishQwen id
" A"	362	337
" B"	425	382
" C"	356	357
" D"	422	434

Scoring this way has a nice side effect: a model that has never seen a multiple choice question still gets graded, because every model assigns *some* probability to every token. We'll never have to parse a rambling answer from a model trained on Shakespeare, which is lucky (it would probably reply in iambic pentameter).

Unfortunately, before irishQwen can assign probabilities to anything, the prompt has to fit inside it.

The Context Window

Qwen3's tokenizer cuts our prompt into 50 tokens, mostly one per word, and its window holds 32 thousand. irishQwen's byte-level BPE tokenizer only learned 512 tokens, all from Shakespeare, so words like "detection" and "distribution" come out in pieces. The same prompt is 143 irishQwen tokens:



We trained irishQwen with a **context window** of 128 tokens, and the slot after `Answer:` needs one of them, so it can only see the last 127 tokens of the prompt. The first 16, greyed out above, fall off the front. irishQwen never sees the words "The following are multiple choice questions". **It is answering a question it can only partly read.**

That's not a bug in the harness, it's just what a 128-token model is. Qwen3, on the other hand, sees every word.

irishQwen's Answer

Here are Shakespeare irishQwen's probabilities for the four letters on our question, as a percentage of all `512` tokens:

```python
import torch

# P(next token) for " A", " B", " C", " D", in % of
# all 512 tokens (Shakespeare irishQwen, our item)
raw = torch.tensor([0.48, 0.59, 0.55, 0.31])
total = round(raw.sum().item(), 2)
print("all four letters:", total, "%")
print("picks:", "ABCD"[raw.argmax()])
```

OUTPUT

```
all four letters: 1.93 %
picks: B
```

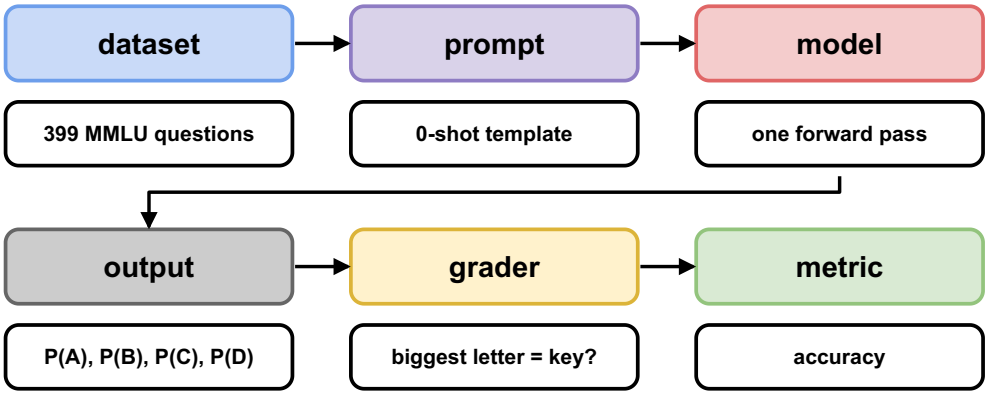
Together the four letters get less than `2%` of the probability. irishQwen mostly expects something else entirely after `Answer:` (which is fair, Shakespeare rarely wrote multiple choice exams). The harness doesn't care. Shared out among the four letters, those numbers (before rounding) become `25.0%`, `30.6%`, `28.4%` and `16.0%`, and sharing them out never changes which one is biggest.

irishQwen picks B. The answer was A, so **irishQwen gets this one wrong**.

One question tells us almost nothing (a random guesser gets it right a quarter of the time). We have to repeat this for every question in the sample and count how many it gets right, the model's **accuracy**.

The Harness

Look at everything we had to decide before we got that one wrong answer. Which questions, how to write them into a prompt, how to run the model, what to read from its output, how to grade it, and what to count. Together those choices are called the **harness**:



Every one of those stages is a knob. A harness could instead let the model write `256` tokens and take the first letter it finds, or sample at a temperature instead of picking the most likely token, or put example questions in the prompt. **The same model on the same questions gets a different score under a different harness**, so a score without its harness doesn't mean much.

The knob that moves scores the most is usually the prompt.

Few-Shot Prompts

Our prompt asks the question cold. MMLU is usually run **5-shot** instead: five solved questions from the same subject (taken from a separate dev split) go in front of ours, each ending in `Answer:` and its letter, so the model sees the pattern before it has to follow it. Ours is called **0-shot**.

Five examples make the prompt much longer:

	characters	Qwen3 tokens	irishQwen tokens
0-shot	244	50	143
5-shot	2,528	622	1,426

At `1,426` tokens the 5-shot prompt is `11.1` times irishQwen's window, so irishQwen would see only the end of the last example and our question. Few-shot prompting just doesn't exist for a model this small.

For Qwen3 it matters a great deal. Our 0-shot harness gives Qwen3-0.6B-Base `39.6%` on our sample. Qwen3's own technical report gives it `52.8%` on MMLU, 5-shot. **Same model, about thirteen points apart**, and the main difference is five examples in the prompt (and the report used all of MMLU, not our `399`). Whenever you see a score, look for the harness next to it.

Three Models, One Harness

Now we can do what benchmarks are for, which is comparing models. We ran the same harness (the same `399` MMLU questions, 0-shot, next-token scoring) on three models:

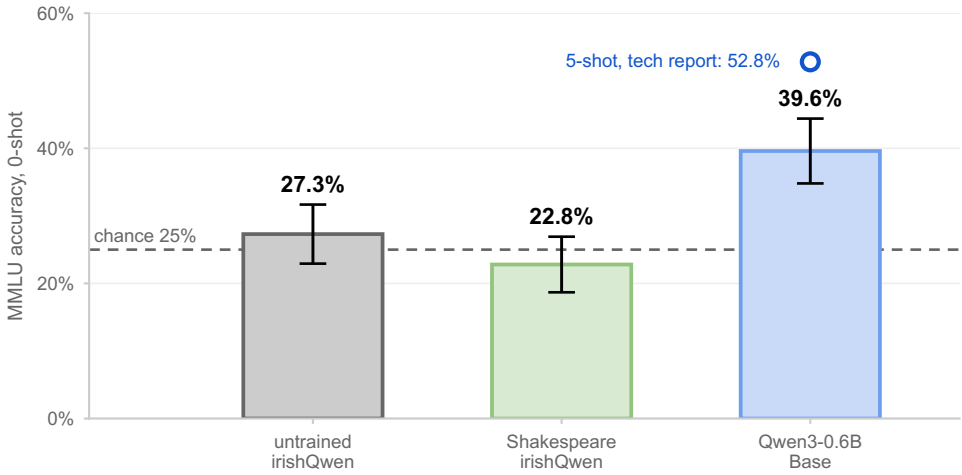
	untrained irishQwen	**Shakespeare irishQwen**	**Qwen3-0.6B-Base**
parameters	3.4 M	3.4 M	0.6 B (28 layers)
trained on	nothing (random weights)	≈ 13 M tokens of Shakespeare	≈ 36 T tokens of web, code and math
vocabulary	512 tokens	512 tokens	151 k tokens
window	128 tokens	128 tokens	32 k tokens

The first two are the same network before and after last chapter's training. The third is Qwen3-0.6B-Base, the smallest of the real Qwen3 models (irishQwen copies its architecture, just narrower and shallower). "Base" means it has only been pretrained, like irishQwen: it continues text, and nobody has taught it to answer questions.

The untrained model is there on purpose. **A score only means something next to a baseline**, and a model with random weights is the most honest baseline there is.

Accuracy

Here's how the three models did:



Qwen3 gets `39.6%` right. Both irishQwens sit right around the `25%` line a random guesser would get, and, awkwardly, **the untrained model beats the trained one**, `27.3%` to `22.8%`.

Did training on Shakespeare make irishQwen worse at machine learning trivia? Before we come up with a story for it, we should ask whether a gap of four and a half points on `399` questions is even real. That's what the black bars on each column are for.

Error Bars

Our 399 questions are a sample. A different 399 MMLU questions would have given each model a slightly different score, the same way flipping a coin 100 times doesn't give exactly 50 heads. How much a score would wobble is its **standard error**:

$$SE = \sqrt{\frac{p(1-p)}{N}}$$

where p is the accuracy and N the number of questions. About 95% of the time the true accuracy lies within 1.96 standard errors of the measured one, the model's **95% interval**:

```python
import torch

n = 399                                      # questions
acc = torch.tensor([0.273, 0.228, 0.396]) # 0-shot
se = torch.sqrt(acc * (1 - acc) / n)
print("±95%:", (1.96 * se * 100).round(decimals=1))
```

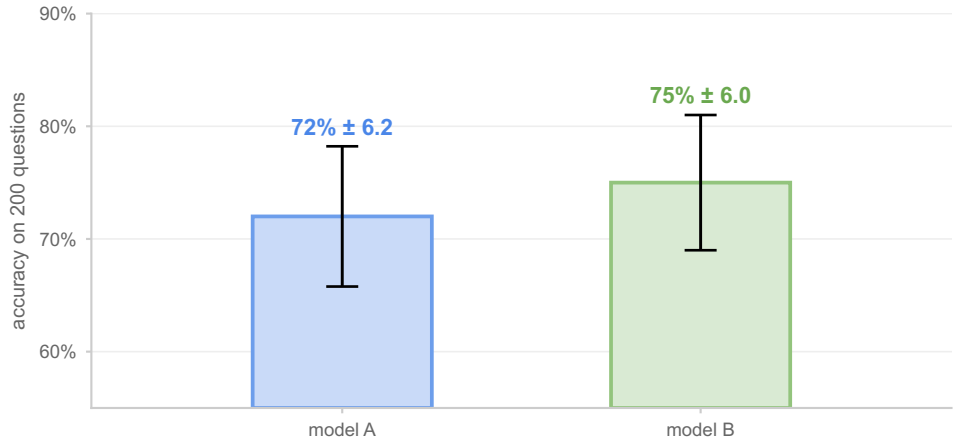
OUTPUT

```
±95%: tensor([4.4000, 4.1000, 4.8000])
```

So Shakespeare irishQwen's 22.8% really means "somewhere between about 18.7% and 26.9%", and the untrained model's 27.3% means about 22.9% to 31.7%. Those intervals overlap almost completely. **Our two irishQwens are indistinguishable on MMLU.**

How Many Questions?

The same arithmetic explains why benchmarks want thousands of questions. Say model A scores `72%` and model B `75%` on the same `200` questions:



Each interval is about ±`6` points wide, and the gap between them is `3.0` ± `8.6` points. **B might not be better at all.** With twenty-five times as many questions (`5,000`) the intervals shrink to about `±1.2` points each, since the standard error falls with the square root of N.

A leaderboard that ranks models by fractions of a point without intervals is ranking noise some of the time.

Better Than Chance?

The more useful question isn't whether two irishQwens differ, it's whether either of them knows anything at all. A random guesser scores 25%, with a standard error of its own. We can count how many of those standard errors each model sits from chance:

```
chance = 0.25
se_chance = (chance * (1 - chance) / n) ** 0.5
z = (acc - chance) / se_chance
print("standard errors from chance:", z.round(decimals=2))
```

OUTPUT

```
standard errors from chance: tensor([ 1.0600, -1.0100,  6.7400])
```

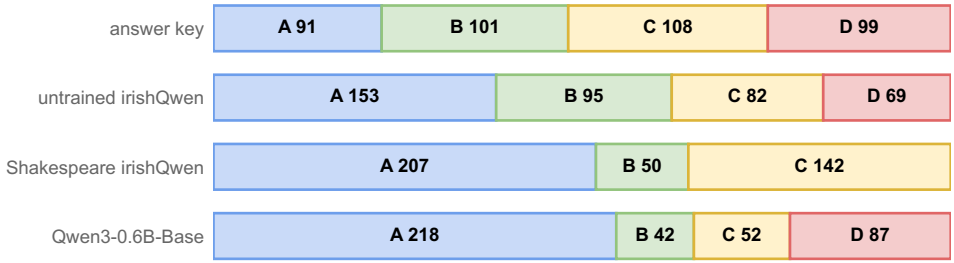
Both irishQwens sit about one standard error from chance, one just above and one just below, which is exactly where a random guesser lands most of the time. Qwen3 is almost seven standard errors above it. A pure guesser essentially never does that.

So the honest summary is that **irishQwen, trained or not, is at chance on MMLU, and Qwen3-0.6B-Base is not**. That's not a knock on our training run. A 3.4-million-parameter model that has read nothing but Shakespeare was never going to know what out-of-distribution detection is.

It does raise a question, though. If irishQwen is guessing, how is it guessing?

Which Letter It Picked

Accuracy only counts right answers. Counting which letter each model picked, across all `399` questions, tells us a lot more:



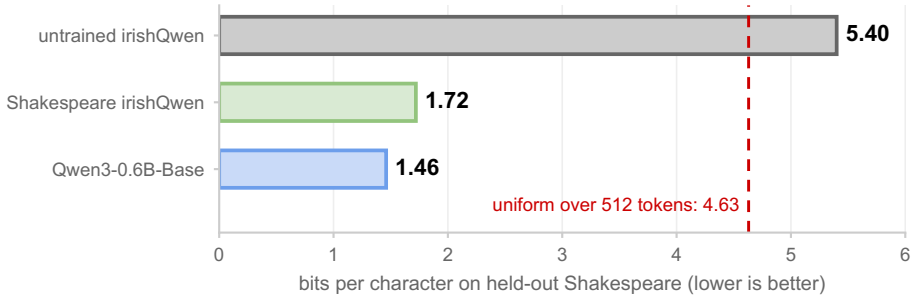
The answer key is close to even. Shakespeare irishQwen is not: it says A `207` times, C `142` times and **never once says D**. It isn't reading the questions, it just likes some letters after `Answer:` more than others (why A and C, we can only guess). The untrained model leans towards A too, more gently.

Notice that a model that ignored the questions completely and always answered C would get `108` of `399` right, `27.1%`, beating Shakespeare irishQwen. **A letter preference alone can move a score by a few points either way.**

Qwen3 has a preference too (A `218` times, against `91` in the key), but it gets far more right, so its answers clearly depend on the question. Shuffling the choices and running again is the usual way to check for this.

Loss Versus Benchmarks

So by MMLU, Shakespeare irishQwen learned nothing. We know that isn't true, because its loss fell so far. Here are all three models on the same held-out Shakespeare passage, in bits per character:



On Shakespeare, our little model is much closer to Qwen3 (`1.72` against `1.46`) than to where it started (`5.40`). Qwen3 is still better, despite never being trained on Shakespeare specifically, because `36` trillion tokens of the internet include a fair amount of Shakespeare (and a lot of English).

The two evals answer different questions. Bits per character asks **how well does it model this kind of text?** MMLU asks whether it knows things. irishQwen is pretty good at the first and hopeless at the second, and neither number alone would have told us that. Picking an eval really means picking the question you want answered.

The Big Five

MMLU is one of a handful of benchmarks that almost every model release in `2026` reports. Each was built to answer a slightly different question, and each grades differently:

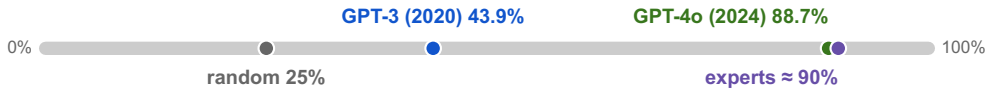
benchmark	tests	format	size	graded by
MMLU / -Pro	broad knowledge	4 or 10 choices	~14k / 12k	exact letter
GPQA Diamond	PhD-level science	4 choices	198	exact letter
AIME	competition math	integer 000–999	30 a year	exact integer
HLE	frontier expertise	short answer + MC	2,500	LLM judge
SWE-bench Pro	real software issues	repo + patch	1,865	hidden tests

Read down the last column and the grading gets more involved: from comparing one letter, to checking a number, to asking another model, to running code. Read down the sizes and notice how small some of them are. With `198` questions, GPQA Diamond's 95% interval near `90%` accuracy is still about `±4` points.

The models these benchmarks are built for have mostly outgrown MMLU, which is the first thing that happens to every benchmark.

Saturation

When MMLU came out in 2020, GPT-3 scored 43.9% on it. Hendrycks et al. estimated that experts in each subject would score around 90%. By 2024, GPT-4o scored 88.7%, and the best models were all bunched up near the top:



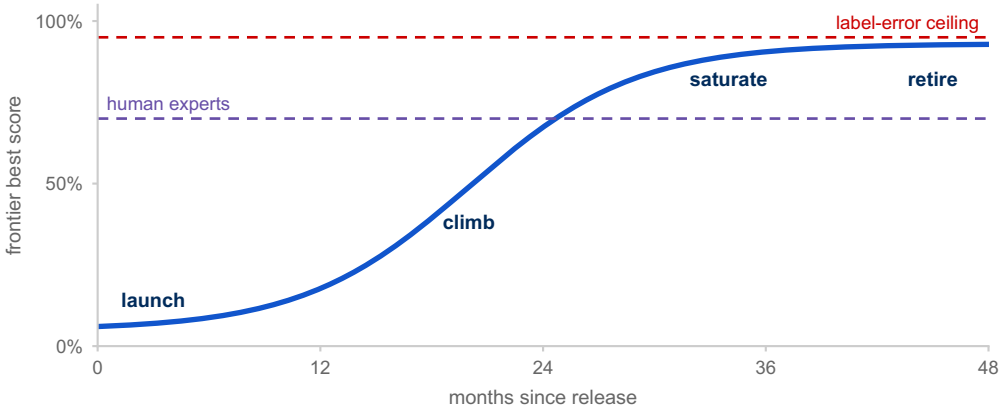
A benchmark everyone scores about the same on is **saturated**: it can't tell the best models apart any more.

It can't go much higher either. When a team re-checked MMLU by hand (MMLU-Redux, 2024) they estimated that 6.49% of its questions contain errors, and 57% of the virology questions. **Past a point, a higher score means agreeing with the mistakes.**

MMLU-Pro (2024) was the fix: harder questions that need more reasoning, with ten choices instead of four, so guessing gets 10% instead of 25% and eliminating the obviously wrong answers helps much less.

Life of a Benchmark

MMLU's story is the usual one. A benchmark launches with the best models near the floor, they climb, the scores bunch up just under whatever ceiling its own label errors allow, and people stop reporting it:

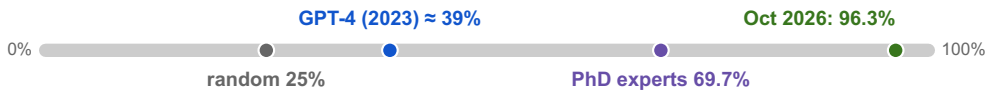


So the newest benchmarks are built to start at the floor on purpose. On ARC-AGI-3, launched in March `2026`, the ARC Prize Foundation reported humans at `100%` and frontier AI at `0.51%`. GPQA Diamond, AIME, HLE and SWE-bench Pro were each built to stay ahead of that curve a little longer.

GPQA Diamond

If models can look up (or have memorized) the answer to an MMLU question, the obvious fix is questions you can't look up. **GPQA** (Rein et al., 2023) is 448 biology, chemistry and physics questions written by PhD experts. It's called "Google-proof" because skilled non-experts with unrestricted web access, spending over half an hour per question, scored only about 34%.

The **Diamond** subset keeps the 198 questions where the expert validators agreed with the answer and most non-experts got it wrong. A panel of PhD experts recruited by OpenAI scored 69.7% on it.



GPT-4 scored about 39% when it came out. As of October 2026, the top score on Artificial Analysis's independent leaderboard was 96.3%. **Google-proof turned out not to be model-proof.** Diamond is close to saturated, and with only 198 questions, a few of them flawed, the top few points barely mean anything.

AIME

The American Invitational Mathematics Examination is a real high school competition: fifteen problems in three hours, every answer an integer from 000 to 999. Two exams a year gives 30 problems, so one problem is worth 3.3 points of score.

Integer answers make grading look easy. It isn't quite. Ask a model for the remainder when 2^{2026} is divided by 1000 and it might write any of these:

model output	strict match	extract and normalize
864	✓	✓
The answer is 864.	✗	✓
\boxed{864}	✗	✓
864.0	✗	✓
984	✗	✗

Four of those are right, and a strict grader marks three of them wrong. **The grader is part of the harness too.**

Because the problems are published every year, only the newest exam is safe from having been trained on, and every year a new one is needed. GPT-4o scored about 13% on AIME 2024. On AIME 2026, the top models score 99 to 100% (as of October 2026).

Humanity's Last Exam

Humanity's Last Exam (HLE, from the Center for AI Safety and Scale AI, January 2025) took the opposite approach to building a hard benchmark. Experts wrote questions, and a question was kept only if the frontier models of the day got it wrong. The final exam has 2,500 questions, 41% of them math, 14% needing an image, and about three quarters with a short exact answer rather than choices.

Short answers can't be graded by comparing letters, so **an LLM judge** reads each response and decides whether its final answer matches. HLE also asks every model how confident it is, and reports **calibration error**: how far the stated confidence is from how often it's actually right. A model that says 92% and is right half the time is badly calibrated.



By construction, every model scored poorly at launch (all under 10%). On Scale's leaderboard, which runs every model on all 2,500 questions itself, the top score was 54.8% as of October 2026. **HLE is still climbing.**

SWE-bench Pro

The last of the five isn't a question at all. **SWE-bench Pro** (Scale AI, September 2025) gives a model an issue from a real code repository (something like "thousands separators should be accepted") and the repository itself, and asks for a patch. Nobody reads the patch. Hidden tests decide: a new test written for the issue has to go from **fail to pass**, and the repository's existing tests have to go from **pass to pass**, so the fix can't break anything that worked.

It has 1,865 tasks from 41 repositories, split into 731 public, 858 held out and 276 from private commercial code. The private split exists so the tasks can't leak into anyone's training data.



GPT-5 solved 23.3% of the public tasks at launch. On Scale's own leaderboard, where every model runs inside the same agent scaffold, the top public score was 61.5% as of October 2026. Labs reporting from their own scaffolds claim much more, which is the harness problem again. And in July 2026, OpenAI estimated that **about 30% of the public tasks were broken** (tests that reject working fixes, tests that demand details the issue never asks for, misleading descriptions), and withdrew the recommendation it had made five months earlier.

Contamination

SWE-bench Pro replaced **SWE-bench Verified**. On February 23, 2026, OpenAI announced it had stopped reporting Verified scores, for two reasons. Of 138 problems it audited by hand, 59.4% had flawed tests or descriptions. And **every frontier model it tested had seen some of the problems in training**, and some could write out the original fix word for word.

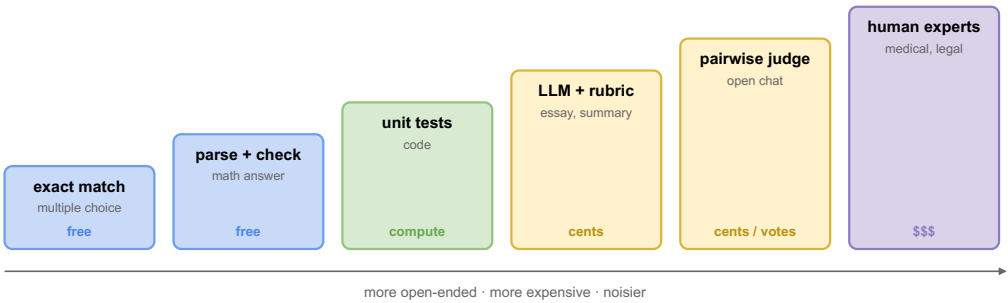
That second problem is **contamination**: test questions (or their answers) end up in the training data, usually because they're on the web. A contaminated model isn't solving the problem, it's remembering it. Labs check for it with:

- **n-gram overlap:** search the training data for test text.
- **Canary strings:** a unique string in every benchmark file.
- **Fresh rewrites:** paraphrased or newer questions.
- **Memorization probes:** can it finish a gold answer?

irishQwen is the one model in this chapter we can promise hasn't seen MMLU, because we know exactly what it read. MMLU's test file has been on the internet since 2020 and Qwen3 read 36 trillion tokens of the internet, so **we can't rule it out for Qwen3**.

Who Grades?

We've now seen four different graders: a letter compared with a key, a number parsed out of text, an LLM judge, and hidden tests. The rule is to use **the cheapest grader that can still tell right from wrong**:



Further right, answers are more open-ended and grading costs more and gets noisier. An essay has no answer key, so someone (or something) has to judge it. LLM judges are cheap, but they have known habits:

bias	what it does	usual fix
position	prefers whichever answer comes first (or second)	swap the order, average
verbosity	longer answers look better	control for length
self-preference	likes answers from its own family	a judge from another lab
drift	the judge model gets updated	pin the judge version

Position bias should sound familiar. It's what Shakespeare irishQwen did with A.

Arena Ratings

For open-ended chat there's often no right answer at all, only answers people like better. **LMArena** shows a person two anonymous models' replies to the same prompt and asks which is better. Every vote is a little match, and models get ratings the way chess players do. The **Bradley–Terry** model behind the ratings says the gap Δ between two ratings sets the chance of winning a vote:

$$P(\text{A beats B}) = \frac{1}{1 + 10^{-\Delta/400}}$$

People also tend to vote for longer, better-formatted answers, so Arena fits length and markdown as extra terms in its model (it calls this **style control**), and a rating tracks what was said more than how it was laid out.

Close Ratings

A vote is a coin flip weighted by the rating gap, so the error-bar arithmetic from earlier tells us how many votes it takes before a win rate is clearly above `50%`:

```python
import torch

gap = torch.tensor([10.0, 50.0, 100.0, 200.0])
p_win = 1 / (1 + 10 ** (-gap / 400))
# votes until the gap is bigger than its 95% interval
votes = (1.96 * 0.5 / (p_win - 0.5)) ** 2
print("A wins:", (100 * p_win).round(decimals=1))
print("votes: ", votes.round())
```

OUTPUT

```
A wins: tensor([51.4000, 57.1000, 64.0000, 76.0000])
votes:  tensor([4640.,  188.,   49.,   14.])
```

A `100`-point gap means A wins `64.0%` of votes, and about `49` votes tell them apart. A `10`-point gap needs thousands. In mid-October `2026` the top five text models were within `21` points of each other, and four of them within `6`, so **the top of the leaderboard is mostly a tie**. It takes a lot of votes to separate them, and a few hundred extra votes from people who like one model's style can reorder them.

pass@k and pass^k

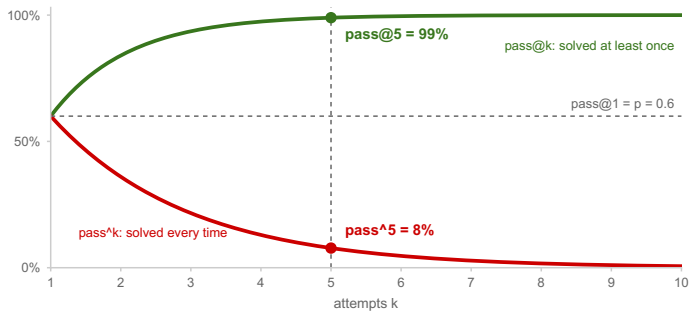
So far each model got one try per question. For code, where tests can check an answer automatically, benchmarks often allow k tries. If one try succeeds with probability p, there are two ways to score k of them:

```python
import torch

p = 0.6                          # one attempt succeeds
k = torch.arange(1, 11)          # 1 to 10 attempts
pass_at_k = 1 - (1 - p) ** k     # at least one works
pass_hat_k = p ** k              # every one works
print("pass@5:", round(pass_at_k[4].item(), 3))
print("pass^5:", round(pass_hat_k[4].item(), 3))
```

OUTPUT

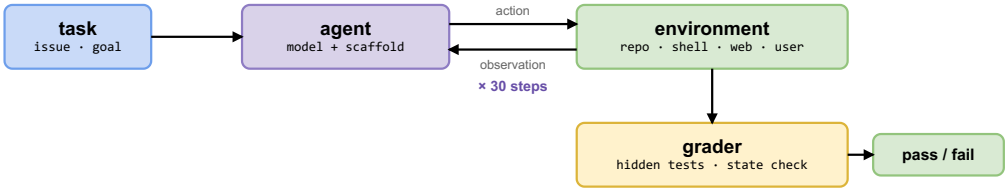
```
pass@5: 0.99
pass^5: 0.078
```



pass@k asks if *any* of the k tries worked, **pass^k** if *all* of them did. At **60%** per try, five tries give **99%** or **8%**, depending on which one you report. pass@k makes sense when a test runner can pick out the good attempt. **If a person is going to rely on the model every time, pass^k is the honest one.**

Agentic Evals

pass^k matters most for agents. SWE-bench Pro was the first agentic eval we've seen: the model doesn't answer once, it works for many steps, reading files, running commands and seeing what happened, and only the end state is graded:



Terminal-Bench (shell tasks in a container), τ-bench (tool use with a simulated customer) and OSWorld (a whole desktop computer) work the same way. The model is also never alone: the tools, retries and step budget around it (the **scaffold**) change the score, so **model and scaffold are measured together**.

Long Tasks

Long tasks are unforgiving. If every step goes right with probability q, all n steps go right q^n of the time:

```python
import torch

q = torch.tensor([0.90, 0.97, 0.99])  # one step right
for steps in [1, 10, 30, 100]:
    print(steps, "steps:", (q ** steps).round(decimals=3))
```

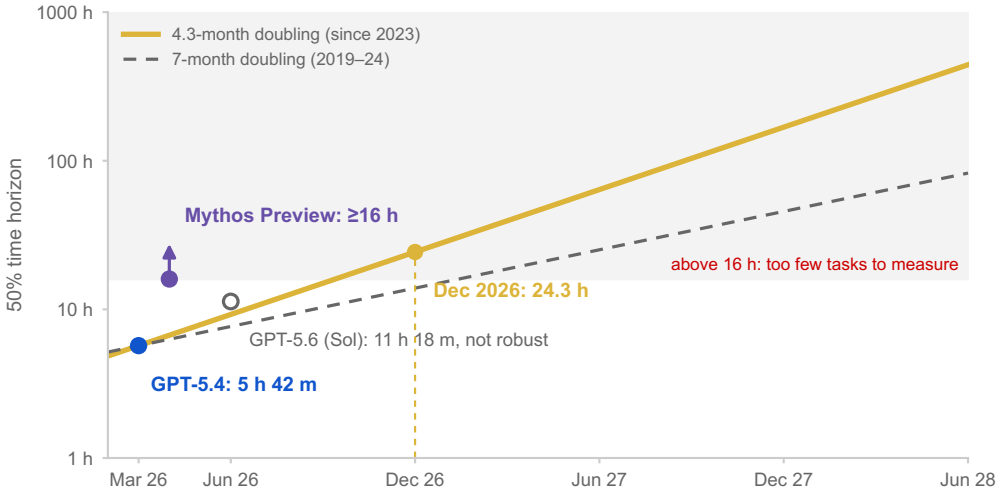
OUTPUT

```
1 steps: tensor([0.9000, 0.9700, 0.9900])
10 steps: tensor([0.3490, 0.7370, 0.9040])
30 steps: tensor([0.0420, 0.4010, 0.7400])
100 steps: tensor([0.0000, 0.0480, 0.3660])
```

A model that gets each step right **97%** of the time finishes a **30**-step task only **40%** of the time. **Small per-step gains turn into big task-level gains**: going from **97%** to **99%** per step takes a **30**-step task from **40%** to **74%**. That's why agentic scores can jump so much between model versions, and why pass^k, not pass@k, is the number to watch for an agent you want to leave alone.

Time Horizons

Agentic scores still saturate like any benchmark. **METR** found a way to put agents from different years on one scale: time the tasks with human experts, then find the task length a model finishes `50%` of the time. That length is the model's **time horizon**.



Claude Mythos Preview, released in April `2026`, measured "at least `16` hours", because **METR's task suite doesn't have enough longer tasks** to measure past that (only `5` of its `228` tasks take an expert that long). GPT-5.6 (Sol), in June `2026`, measured `11` h `18` min, and METR said even that wasn't a robust number, because Sol kept finding loopholes in the tasks (counting those as successes would have put it above `270` hours).

Doubling Times

What made time horizons famous is how fast they grow. In March 2025 METR reported horizons doubling about every 7 months from 2019 to 2024. In January 2026 (Time Horizon 1.1) it put the doubling time since 2023 at about 4.3 months. The two lines in the last figure are those two rates.

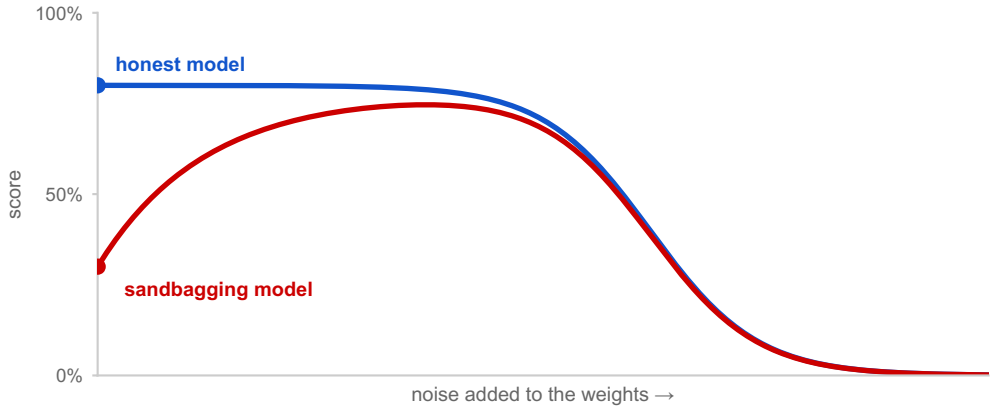
There are three good reasons to read the line carefully. Mythos Preview's 80% horizon (tasks it finishes four times out of five) was only 3 h 6 min, so **"50%" hides a lot of failure**. The tasks are software and research engineering, not work in general, and other kinds of work may trend differently.

And the doubling is a trend, not a law. Sol's number can't anchor a forecast, so we start from the most recent robust measurement on METR's page (as of September 8, 2026): GPT-5.4, released in March 2026, at 5 h 42 min. Doubling every 4.3 months, December 2026 would be 24.3 hours (about 3 working days), but only if the trend holds. **The answer changes a lot with the doubling time you assume**: at 7 months it would be about 14 hours.

Eval Awareness

Every eval in this chapter assumes the model does its best. Unfortunately, a model that can tell it's being tested doesn't have to. Anthropic reported that Claude Sonnet `4.5` recognized many of its alignment tests as tests, saying so about `13%` of the time ("I think you're testing me", in one case). This is **eval awareness**.

A model that deliberately underperforms when a high score would get it restricted is **sandbagging**. One way to catch it (Tice et al., `2024`) is to add noise to the weights. An honest model only gets worse. A sandbagging model often gets *better* at first, because the noise breaks the holding back before it breaks the skill:



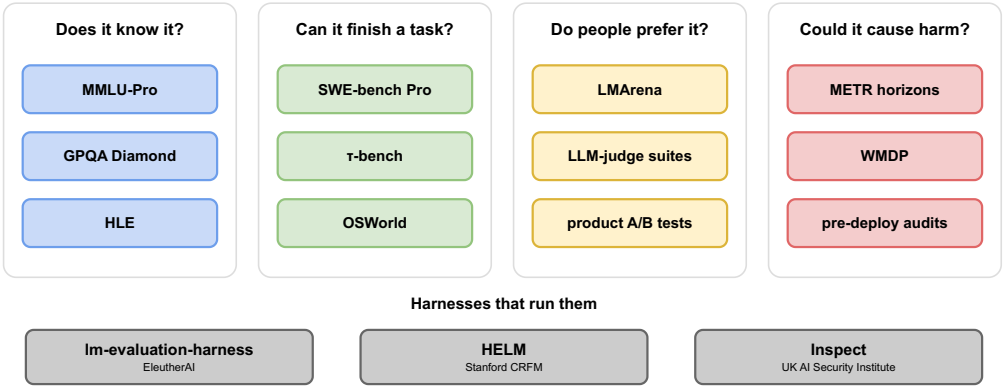
Alignment Faking

The same worry applies to training. In **alignment faking** (Greenblatt et al., December 2024), Claude 3 Opus complied with harmful requests 14% of the time when told its answers to free users would be used to retrain it, and almost never for paid users. Realistic tasks, honeypots, reading the model's reasoning and probing its activations are all used against this. **A score is a measurement of a model that may know it's being measured.**

None of this applies to our irishQwen, which can't tell a test from a sonnet. It matters more the more capable a model gets, which is exactly when the evals matter most.

Which Eval Answers What?

There's no single number for how good a model is, only answers to particular questions. Each family of evals answers one:



Our three models only ever faced the first column, and even that needed a trick. We scored Qwen3-0.6B-Base by reading letters off its next-token probabilities because it's a base model: it continues text, it doesn't answer it. Ask it a question and it's as likely to carry on with another question as to reply.

Nearly every model on these leaderboards is something else, an assistant that answers when asked. **Getting from a base model to an assistant** takes more training, and the first step of that is showing the model examples of what an answer looks like.